

The CrowdStrike Global IT Outage (2024)

Suresh L. Paul

2026-03-11

Introduction

On July 19, 2024, the world woke up to one of the largest IT outages in history. Airlines cancelled thousands of flights. Hospital systems went offline. Banks could not process transactions. Emergency services scrambled to manage without their usual tools. The culprit was not a cyberattack or a natural disaster — it was a single software update pushed out by a cybersecurity company called CrowdStrike.

This case is a powerful reminder that in our hyper-connected world, a small mistake in the right (or wrong) place can have catastrophic consequences across the globe.

Background: Who Is CrowdStrike?

CrowdStrike is a cybersecurity company — its job is to protect computers and networks from hackers, viruses, and other digital threats. Think of it as a high-tech security guard for your computer systems.

One of their flagship products is called **Falcon**, a security software that runs quietly in the background on millions of computers worldwide. Large organizations — airlines, hospitals, banks, government agencies — pay CrowdStrike to have Falcon installed on their systems to keep them safe.

Falcon works by constantly monitoring a computer for suspicious behavior. To do its job effectively, it needs to be kept up to date. The digital threat landscape changes daily, and the software needs new “rules” to recognize the latest threats. These updates are called **content configuration updates**, and CrowdStrike pushed them out automatically and frequently — often multiple times a day.

What Happened?

On the morning of July 19, 2024, CrowdStrike released a routine content update to its Falcon software for computers running Microsoft Windows. The update contained a flawed file — essentially a set of instructions that Falcon could not properly read.

When computers downloaded this update, Falcon crashed. And because Falcon runs at a very deep level of the operating system (the core software that controls everything a computer does), when Falcon crashed, it took the entire computer down with it.

The result was the **Blue Screen of Death** — the dreaded blue error screen Windows displays when something has gone catastrophically wrong. Affected computers could not restart properly. They were stuck in a loop, crashing over and over again.

Key facts about the outage: - Approximately **8.5 million Windows devices** were affected worldwide - The outage hit **airlines, hospitals, banks, stock exchanges, broadcasters, emergency services, and retail chains** - Delta Air Lines alone cancelled over **7,000 flights** and reported losses of over **\$500 million** - Hospitals in multiple countries were forced to cancel non-emergency surgeries and revert to paper records - The fix required IT teams to manually restart each affected computer and apply a patch — a process that could not be automated at scale and took **days** to complete at large organizations

Why Was the Fix So Difficult?

You might wonder: if a software update caused the problem, why not just push out another update to fix it?

The problem was that the affected computers could not start up properly in the first place. It's like trying to fix a car engine that won't turn on — you can't drive to the mechanic.

IT teams had to physically (or remotely, where possible) access each broken machine, boot it into a special recovery mode, manually delete the problematic file, and then restart it. For a company with 10,000 computers, this meant 10,000 individual interventions. For companies with global operations, this meant coordinating efforts across continents and time zones.

What Went Wrong? An MIS Perspective

1. Inadequate Software Testing

Before any software update is released to the public, it should go through rigorous **testing** — running it in a controlled environment to catch errors before they affect real systems. Industry best practice is to test updates on a small sample of machines first, then gradually roll out to more users (this is called a **staged rollout** or **canary deployment**).

CrowdStrike's update was pushed directly to all users simultaneously without adequate testing. The flawed file apparently passed automated checks but was never caught because the testing process had a gap.

2. The Risk of Automatic Updates

Automatic updates are convenient and generally a good thing — they keep software current without requiring users to do anything. But they also mean that a flawed update can spread everywhere instantly. There is no human checkpoint in the middle saying “wait, does this look right?”

This is a fundamental tension in IT management: **speed vs. control**. Pushing updates fast is important in cybersecurity (threats emerge quickly), but fast also means less time to catch mistakes.

3. Single Point of Failure

A **single point of failure** is any component of a system that, if it breaks, causes the entire system to stop working. CrowdStrike's widespread adoption meant that when Falcon failed, it failed everywhere at once. Organizations that had standardized on a single security vendor had no backup.

This raises an important question for IT managers: **Is it wise to rely on one vendor for a mission-critical function?**

4. Change Management

In IT, **change management** refers to the process of carefully planning, reviewing, approving, and monitoring changes to systems. Good change management asks: What could go wrong? How do we detect a problem early? How do we roll back if something breaks?

The CrowdStrike incident suggests that the change management process for content updates was less rigorous than for full software releases — perhaps because content updates were considered “routine” and lower risk.

Aftermath and Response

CrowdStrike's CEO, George Kurtz, issued a public apology and the company published a detailed **Root Cause Analysis (RCA)** — a document explaining exactly what went wrong and how. This kind of transparency is considered good practice in IT incident management.

The company pledged to: - Improve its testing procedures - Introduce staged rollouts for content updates - Give customers more control over when and how updates are applied

Regulatory bodies in several countries launched investigations. U.S. Congressional committees summoned CrowdStrike executives to testify. Delta Air Lines announced it was pursuing legal action to recover its losses.

The outage also reignited a broader debate in the technology industry about the risks of **software monoculture** — the danger of so many organizations depending on the same tools from the same vendors.

Key Concepts for Discussion

Concept	Definition	Relevance to This Case
Software Testing	Verifying that software works correctly before release	The flawed update was not caught before release
Change Management	Structured process for managing system changes	Update procedures lacked sufficient controls
Single Point of Failure	One component whose failure brings down everything	CrowdStrike's widespread use amplified the damage
Staged Rollout	Releasing updates gradually, not all at once	Not used for content updates; would have limited damage
Business Continuity	Plans to keep operating when IT systems fail	Many organizations were not prepared to operate manually

Analysis Questions

1. Describe the company featured in the case, including its competitive advantage (or “moat”), historical background, and the nature of its industry or business operations.
2. Summarize the key facts presented in the case. Do these facts relate to or mirror any other cases discussed in class? If so, what insights or lessons from those previous cases can be applied here?
3. Analyze the economic consequences of the information system failure described. If a similar failure occurred in today's technological environment, would the impact be greater or smaller? How likely is such an incident to occur now?
4. Discuss the broader implications of the risks identified in the case for contemporary IT infrastructures, cloud computing services, and emerging artificial intelligence systems.
5. Identify the key human decision point in this case — the moment where a different choice by a person or team would most likely have prevented or significantly reduced the harm. Who was responsible, and what should they have done differently?
6. Identify and explain five key lessons or takeaways derived from this case study.

Discussion Questions

1. What does this case reveal about where organizations tend to underestimate risk — in their technology, their processes, or their people? Use specific evidence from the case to support your answer.
 2. If you were the CIO of the organization in this case, what are the first three changes you would make after this incident, and why?
 3. CrowdStrike’s product was designed to protect organizations, yet it caused massive disruption. What does this tell us about the risks of depending on third-party software vendors for mission-critical functions?
 4. There is a fundamental tension in cybersecurity between pushing updates fast (to stay ahead of threats) and testing thoroughly (to avoid breaking things). How should a company like CrowdStrike manage that trade-off? Who in the organization should own that decision?
 5. Many of the hardest-hit organizations had no effective backup plan when their computers went down. What does a good business continuity plan look like, and why do organizations often neglect it?
 6. Should governments regulate how software updates are tested and deployed for companies whose products touch critical infrastructure? What are the pros and cons?
-

Suggested Further Reading

- CrowdStrike’s official Preliminary Post Incident Review (available at [crowdstrike.com](https://www.crowdstrike.com))
 - U.S. Congressional [testimony](#) by CrowdStrike CEO George Kurtz (July 2024)
 - “The Fragility of Software Infrastructure” — ([Columbia](#), MIT Technology Review)
-

Instructions

1. Use **Bloomberg News** as the primary source for case research. You may also reference other *reputable* public websites or databases. All sources must be properly cited in **APA format** within your written report.
2. The **written case report** should be between **5 and 10 double-sided pages**, inclusive of tables, figures, infographics, or any analytical materials. Acceptable submission formats include **Canva, Microsoft Word, PDF, Markdown, or LaTeX**.
3. The **case presentation** should consist of **10 to 30 slides**, incorporating tables, charts, infographics, or analytical visuals as needed. Acceptable file formats include **Canva, Microsoft PowerPoint, PDF, Quarto Markdown, or LaTeX**.
4. Include a **detailed log of each team member’s contributions** as an appendix to the written report. Every team member must be prepared to address questions during the presentation, both on the overall case content and specifically on the sections they contributed to.
5. **AI tools may not be used** to generate written content for the report. Collaborative work through **Google Docs** (with version control and simultaneous editing) is encouraged to demonstrate authentic team contribution and transparency in authorship. All version control tracking links must be shared with the instructor when asked.

*Prepared for Suresh Paul's MIS Course — Case Study Student Reading. Not intended for commercial use.
Not intended for AI training*